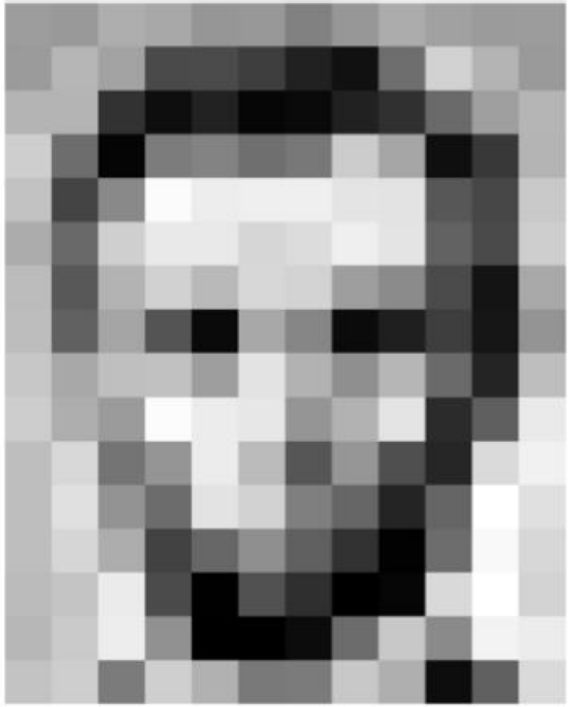


Deep Learning (702AI0C008)

Unit-4 Convolutional Neural Networks

What you see



Input Image

What you both see

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
196	206	123	207	177	121	123	200	175	13	96	218

Input Image + values

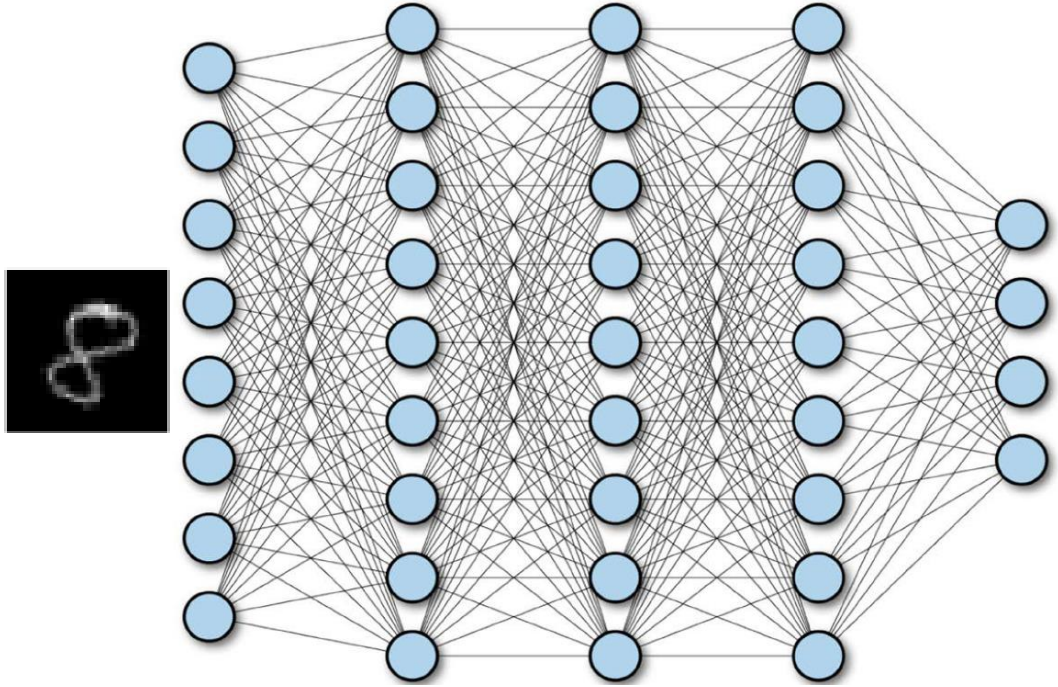
What the computer "sees"

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
196	206	123	207	177	121	123	200	175	13	96	218

Pixel intensity values
("pix-el"=picture-element)

ANN

- In MNIST dataset
  - Images are gray scale of size 28x28x1 pixels
- All neurons of a layer are connected to all neurons of the previous layer in fully connected network
- This may require large number of neurons



Advantages of ANN

- General-purpose: Works with 1D vectors of any kind (tabular data, signals, etc.).
- Simplicity: Easier to implement conceptually; no need for convolution or pooling operations.
- Effective for small-scale problems: Good for simple classification or regression when spatial structure is not important.
- Global connectivity: Every neuron connects to every neuron in the next layer, which can capture non-local relationships.

Drawbacks of ANN

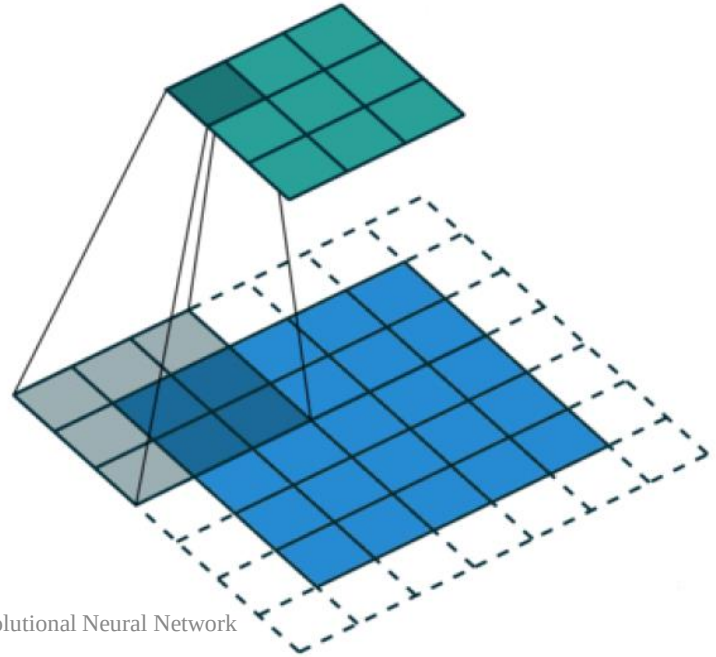
- Ignores spatial structure: Loses relationships between neighboring pixels in images.
- Parameter-heavy: For image inputs (e.g., $28 \times 28 \times 1 \approx 784$ features), fully connected layers explode in parameter count → high memory and compute cost.
- Overfitting risk: So many parameters require more data and regularization.
- Inefficient for high-dimensional inputs: Slow to train and may not generalize well.

Key idea: re-use parameters

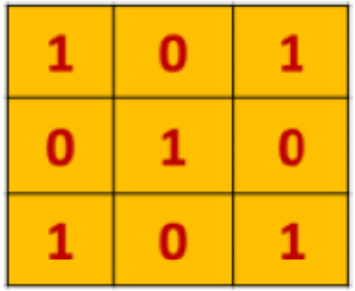
Convolution shares parameters

Example 3x3 convolution on a 5x5 image

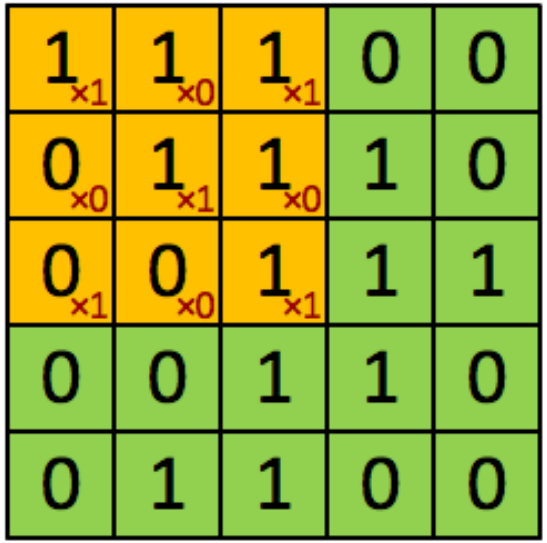
1)Apply a set of weights –a filter – to extract **local features**
2)Use **multiple filters** to extract different features
3)Spatially share parameters of each filter



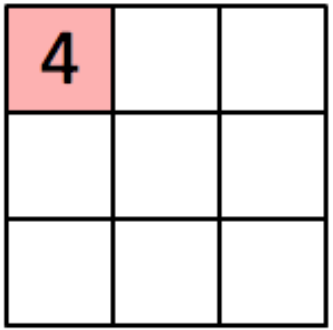
Convolution



Filter / Kernel

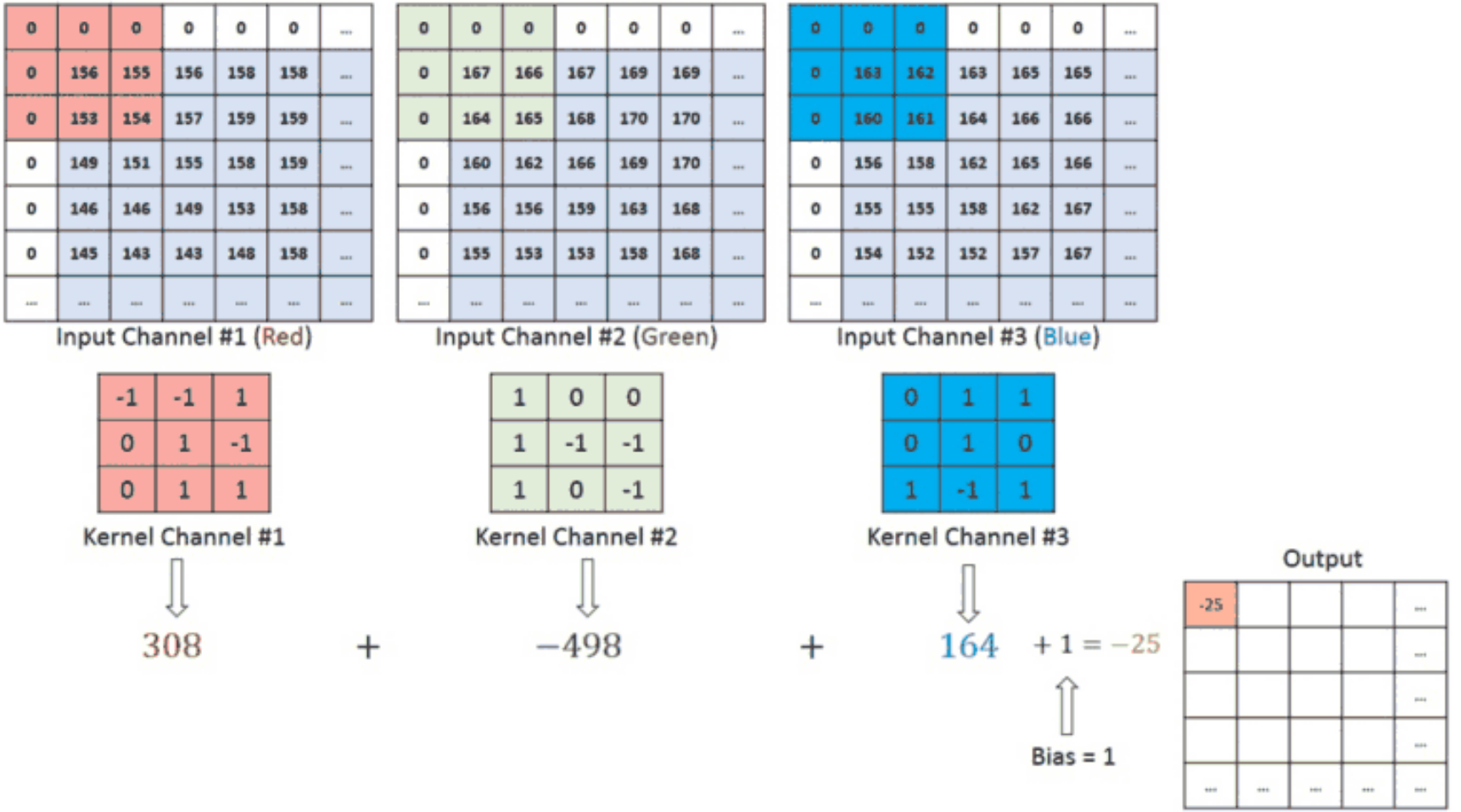


Image



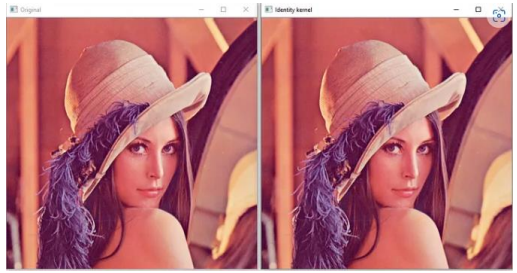
Convolved Feature

Convolution operation is element wise multiply and add

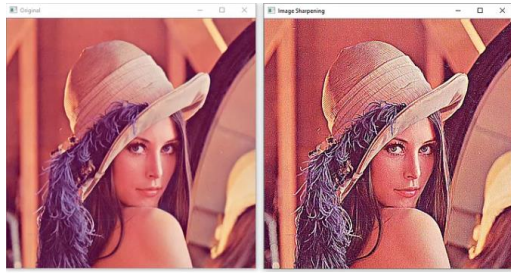

0 0 0 0 0 0 ...
0 156 155 156 158 158 ...
0 153 154 157 159 159 ...
0 149 151 155 158 159 ...
0 146 146 149 153 158 ...
0 145 143 143 148 158 ...
... ...
Input Channel #1 (Red)
0 0 0 0 0 0 ...
0 167 166 167 169 169 ...
0 164 165 168 170 170 ...
0 160 162 166 169 170 ...
0 156 156 159 163 168 ...
0 155 153 153 158 168 ...
... ...
Input Channel #2 (Green)
0 0 0 0 0 0 ...
0 163 162 163 165 165 ...
0 160 161 164 166 166 ...
0 156 158 162 165 166 ...
0 155 155 158 162 167 ...
0 154 152 152 157 167 ...
... ...
Input Channel #3 (Blue)
-1 -1 1
0 1 -1
0 1 1
Kernel Channel #1
1 0 0
1 -1 -1
1 0 -1
Kernel Channel #2
0 1 1
0 1 0
1 -1 1
Kernel Channel #3
308
+
−498
+
164
+ 1 = −25
Bias = 1
Output
-25 ... ...
... ...
... ...
... ...
... ...

Simple Kernels / Filters

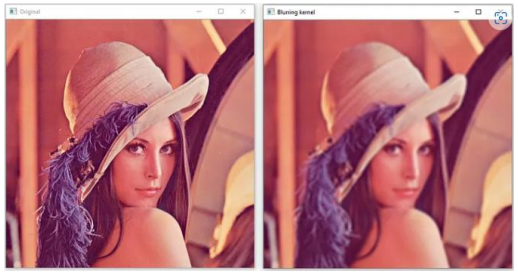
$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

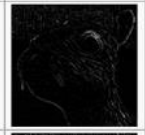



```
[[-1 -1 -1]
 [-1  9 -1]
 [-1 -1 -1]]
```



$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

❑ Conv 3x3 with stride=1,padding=0

1	2	1	1	1	1
1	1	5	3	9	1
2	4	4	7	5	3
3	6	7	5	6	2
1	6	5	3	1	2
2	3	1	1	1	1

6x6 Image



21	28	36	31
33	42	51	41
38	47	43	34
34	37	30	22

4x4

❑ Conv 3x3 with stride=1,padding=1

0	0	0	0	0	0
0	1	5	3	9	0
0	4	4	7	5	0
0	6	7	5	6	0
0	6	5	3	1	0
0	0	0	0	0	0

4x4 Image



14	24	33	24
27	41	42	21
33	32	35	27
24	32	27	15

4x4

❑ Conv 3x3 with **stride=2**,padding=0

1	2	1	1	1	1	1
1	1	5	3	9	1	1
2	4	4	7	5	3	1
3	6	7	5	6	2	2
1	6	5	3	1	2	1
2	3	1	1	1	1	1
1	1	1	3	2	2	1

7x7 Image



14	36	23
38	43	23
21	18	12

3x3

❑ Conv 3x3 with **stride=2**,padding=1

0	0	0	0	0	0	0
0	1	2	1	1	1	0
0	1	1	5	3	9	0
0	2	4	4	7	5	0
0	3	6	7	5	6	0
0	1	6	5	3	1	0
0	0	0	0	0	0	0

5x5 Image



5	13	14
17	42	35
16	32	15

3x3

Output volume/shape

- Input of size W x W x D
  - WxW number of rows and columns
  - D is number of channels
- k number of filters/kernels with a spatial size of F with stride S and amount of padding P, then the size of output volume can be determined by

$$W_{out} = \frac{W - F + 2P}{S} + 1$$

- output volume of size W_{out} x W_{out} x k

- Our convolutional neural network has architecture as follows:
- [INPUT]
- →[CONV 1] → [BATCH NORM] → [ReLU] → [POOL 1]
- → [CONV 2] → [BATCH NORM] → [ReLU] → [POOL 2]
- → [FC LAYER] → [RESULT]

CONV 1

Input Size ($W_1 \times H_1 \times D_1$) = 28 x 28 x 1

- Requires four hyperparameter:
  - Number of kernels, k = 16
  - Spatial extend of each one, F = 5
  - Stride Size, S = 1
  - Amount of zero padding, P = 2
- Outputting volume of $W_2 \times H_2 \times D_2$
  - $W_2 = (28 – 5 + 2 (2)) / 1 + 1 = 28$
  - $H_2 = (28 – 5 + 2 (2)) / 1 + 1 = 28$
  - $D_2 = k$

Output of Conv 1 ($W_2 \times H_2 \times D_2$) = 28 x 28 x 16

POOL 1

Input Size ($W_2 \times H_2 \times D_2$) = 28 x 28 x 16

- Requires two hyperparameter:
  - Spatial extend of each one, F = 2
  - Stride Size, S = 2
- Outputting volume of $W_3 \times H_3 \times D_2$
  - $W_3 = (28 – 2) / 2 + 1 = 14$
  - $H_3 = (28 – 2) / 2 + 1 = 14$

Output of Pool 1 ($W_3 \times H_3 \times D_2$) = 14 x 14 x 16

CONV 2

Input Size ($W_3 \times H_3 \times D_2$) = 14 x 14 x 16

- Requires four hyperparameter:
  - Number of kernels, k = 32
  - Spatial extend of each one, F = 5
  - Stride Size, S = 1
  - Amount of zero padding, P = 2
- Outputting volume of $W_4 \times H_4 \times D_3$
  - $W_4 = (14 – 5 + 2 (2)) / 1 + 1 = 14$
  - $H_4 = (14 – 5 + 2 (2)) / 1 + 1 = 14$
  - $D_3 = k$

Output of Conv 2 ($W_4 \times H_4 \times D_3$) = 14 x 14 x 32

POOL 2

Input Size ($W_4 \times H_4 \times D_3$) = 14 x 14 x 32

- Requires two hyperparameter:
  - Spatial extend of each one, F = 2
  - Stride Size, S = 2
- Outputting volume of $W_5 \times H_5 \times D_3$
  - $W_5 = (14 – 2) / 2 + 1 = 7$
  - $H_5 = (14 – 2) / 2 + 1 = 7$

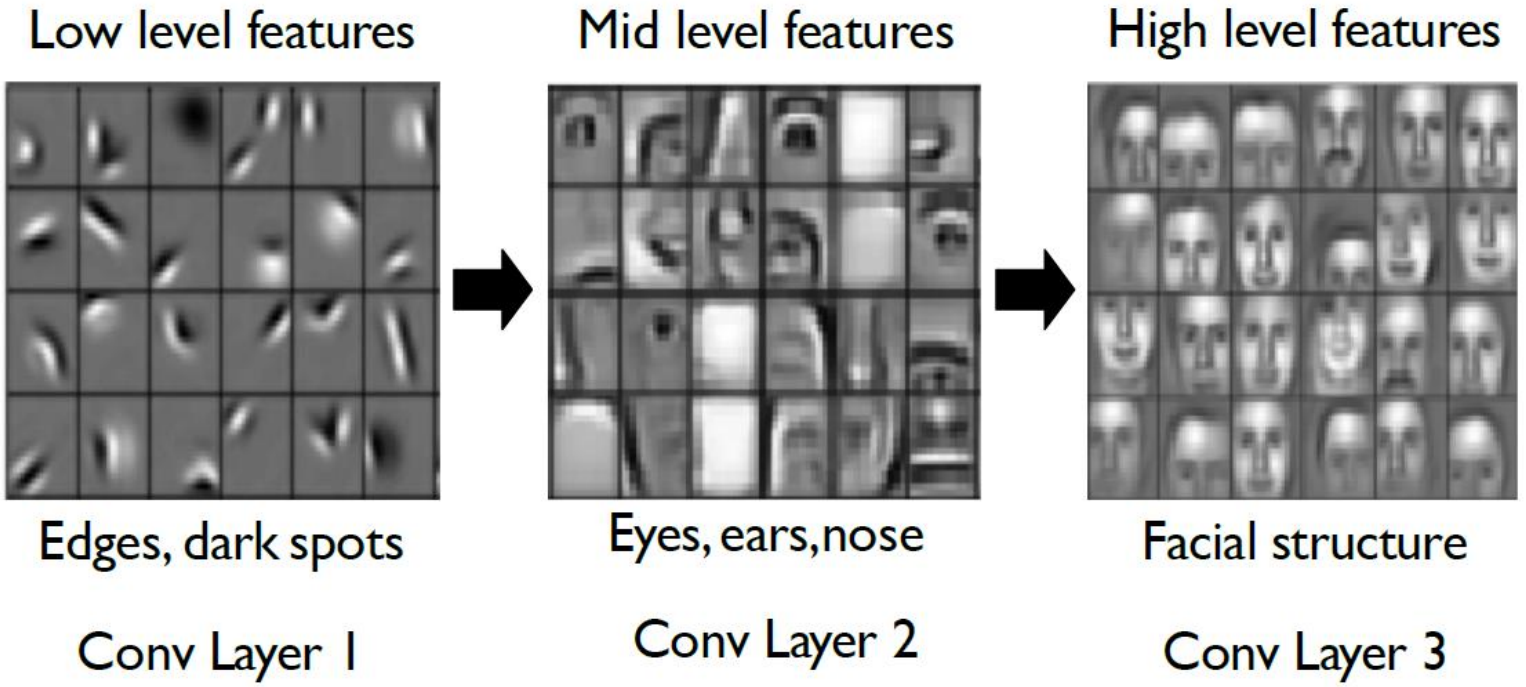
Output of Pool 2 ($W_5 \times H_5 \times D_3$) = 7 x 7 x 32

FC Layer

Input Size ($W_5 \times H_5 \times D_3$) = 7 x 7 x 32

Output Size (Number of Classes) = 10

Key idea: learn hierarchy of features directly from the data(rather than hand-engineering them)



Producing Feature Maps/Activation Maps



Original



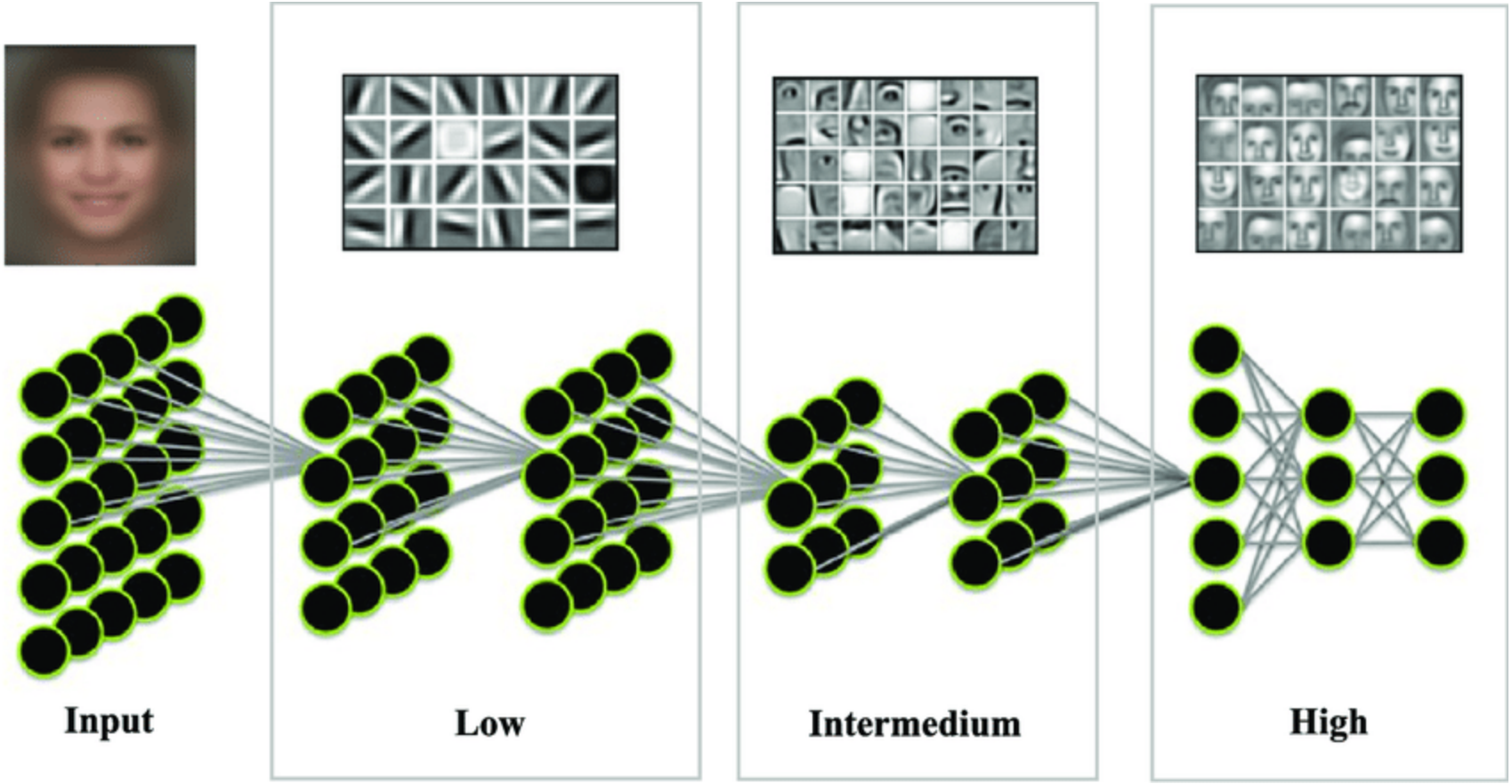
Sharpen



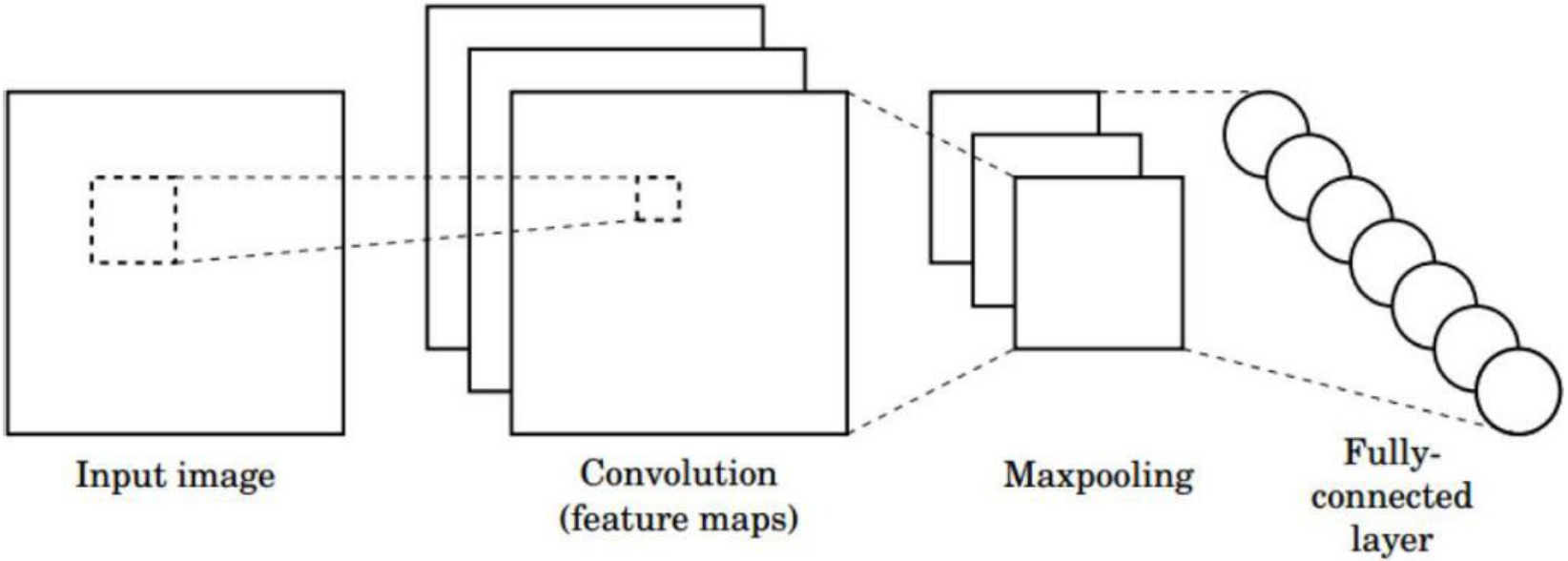
Edge Detect



"Strong" Edge Detect


Input
Low
Intermedium
High

- **Input Layer:** Receives the raw image data, represented here as a blurred face.
- **Low-Level Features:** The first convolution layer extracts basic features like edges and orientations, depicted as small, simple patterns.
- **Intermediate Features:** Subsequent layers combine these basic features into more complex patterns, such as parts of a face (eyes, nose, mouth), shown as more recognizable facial components.
- **High-Level Features:** The final layers represent high-level, abstract features that are highly discriminative for recognizing the entire face, shown as stylized full faces.

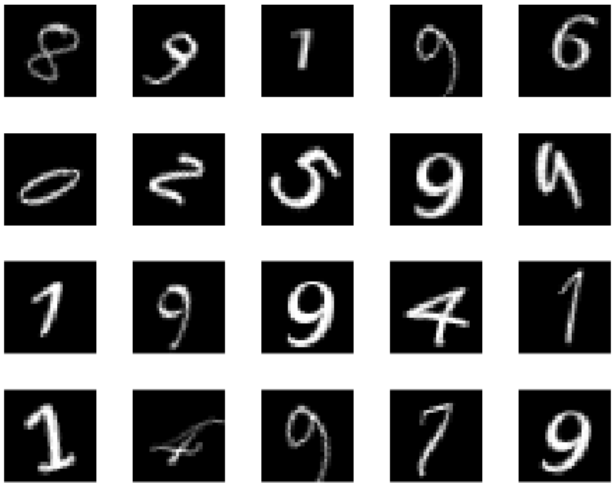

Input image
Convolution
(feature maps)
Maxpooling
Fully-
connected
layer

CNN layers

- A complete Convolution Neural Network architecture is also known as covnets. A covnets is a sequence of layers, and every layer transforms one volume to another through a differentiable function.
- **Input Layers:** It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images.
- **Convolutional Layers:** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images.
- **Activation Layer:** By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are **ReLU**
- **Pooling Layer:** This layer is periodically inserted in the covnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are **max pooling** and **average pooling**.
- **Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.
- **Fully Connected Layers:** It takes the input from the previous layer and computes the final classification or regression task.
- **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.

Data Preparation

- Embark on our journey by gathering a vast collection of images showcasing the objects we want CNN to recognize. In our example, let's say we're building a system to identify different numbers.
- These images undergo pre-processing, ensuring they have a uniform size and format. This allows the CNN to efficiently work with the data.



Feature Extractors/ Convolution

- These layers act like tiny detectives meticulously examining the image. They employ small filters (typically 3x3 pixel grids) to identify specific features within the image.
- Imagine a magnifying glass focusing on a small area of the image.
- As this filter slides across the entire image, it performs a mathematical operation called convolution to detect patterns.
- In early convolutional layers, these filters might be looking for basic shapes like edges, corners, or blobs etc.
- As progress through more layers, the filters become more complex, searching for combinations of these basic features that form higher-level features like eyes, ears, or wings in our animal recognition example.

Activation Functions

- The output from the convolutional layers is then processed by activation layers
- Introduces a non-linearity into the network
- They apply a specific mathematical function (like ReLU) that acts like a threshold, allowing only features exceeding a certain level of importance to pass on.
- Deciding which clues (features) are most relevant to identifying the animal in the image
- Activation layers add complexity and help the network learn more intricate relationships between features
- Allowing neural networks to learn and approximate complex mappings between inputs and outputs.
- Two commonly used activation functions are
- 1. ReLU (Rectified Linear Activation)
- 2. Softmax.

ReLU

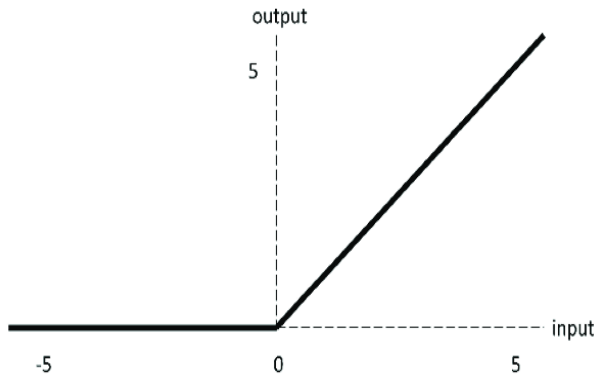
- ReLU is one of the most widely used activation functions in deep learning.
- It is simple and computationally efficient, and it helps alleviate the vanishing gradient problem. ReLU is defined as:

$$f(x) = \max(0, x)$$

- When $x > 0$, the function outputs x.
- When $x \leq 0$, the function outputs 0.

In mathematical terms:

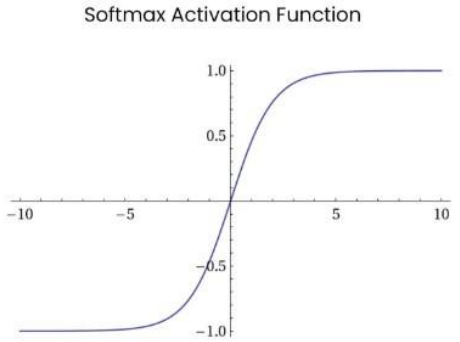
$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$



Softmax

- Softmax is commonly used in the output layer of a neural network when dealing with classification problems
- It converts raw scores (logits) into probabilities, where each class probability is between 0 and 1, and the sum of all probabilities are defined as:

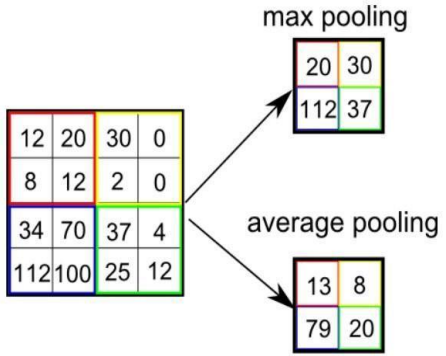
$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$



- For each element x_i *in* the input vector, it calculates the exponential of x_i , then divides it by the sum of the exponentials of all elements in the input vector.
- Characteristics:
  - **Probability Distribution**: Softmax ensures that the output values are normalized into a probability distribution. This is crucial for classification tasks, where the model needs to predict the probability of each class.
  - **Cross-Entropy Loss**: Softmax is often paired with the cross-entropy loss function in classification tasks, as they work well together.

Pooling Layers

- Pooling layers play a crucial role in reducing the dimensionality of the data coming from the convolutional layers.
- Pooling is performed in neural networks to reduce variance and computation complexity
- This helps to:
  - **Manage the number of parameters:** control the number of parameters in the network, making it more efficient and faster to train.
  - **Prevent overfitting:** when the model memorizes the training data too well and performs poorly on new, unseen data.
- It helps to generalize the learned features and improve the network's ability to handle variations.
- Pooling acts like summarizing the key findings from the detective's observations in each region of the image. It retains the most critical information while discarding less important details.
- The three types of pooling operations are:
  - Max pooling: The maximum pixel value of the batch is selected.
  - Min pooling: The minimum pixel value of the batch is selected.
  - Average pooling: The average value of all the pixels in the batch is selected.



❑ MaxPooling 2x2 with **stride=2**

14	24	33	24
27	41	42	21
33	32	35	27
24	32	27	15

4x4 Image



41	42
33	35

2x2

❑ MaxPooling 3x3 with **stride=2**

1	2	1	1	1	1	1
1	1	5	3	9	1	1
2	4	4	7	5	3	1
3	6	7	5	6	2	2
1	6	5	3	1	2	1
2	3	1	1	1	1	1
1	1	1	3	2	2	1

7x7 Image

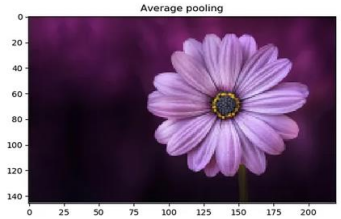


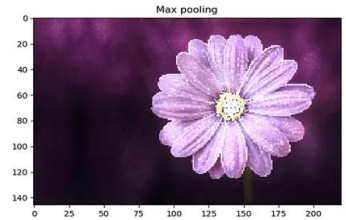
5	9	9
7	7	6
6	5	2

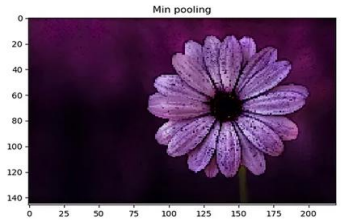
3x3

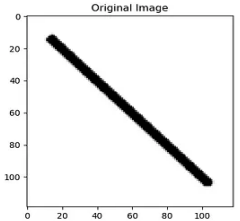
Pooling Layers

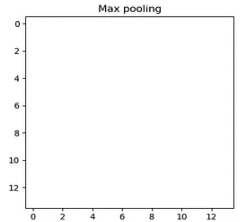




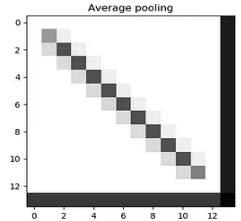


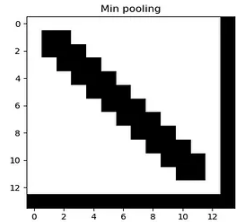




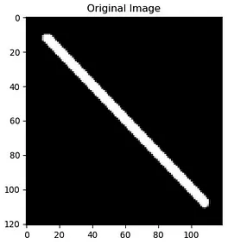


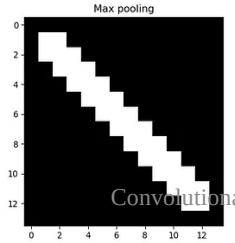
Min pooling gives better result for images with white background and black object



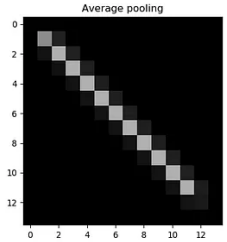


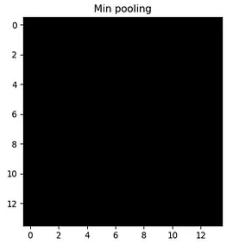
- Average pooling smooths out the image and hence the sharp features may not be identified.
- Max pooling selects the brighter pixels from the image. It is useful when the background of the image is dark.
- Similarly, min pooling is used in the other way round.





Max pooling gives better result for the images with black background and white object (Ex: MNIST dataset)

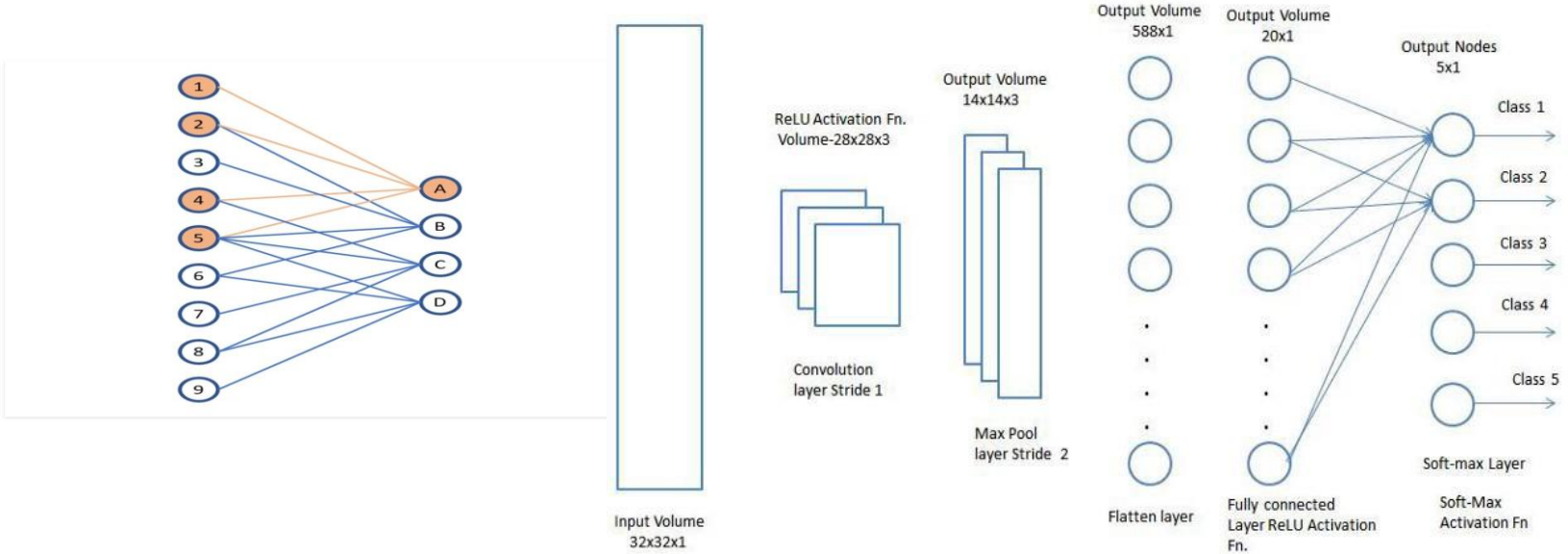




For MNIST digits dataset using CNN, max pooling is used because the background in these images is made black to reduce the computation cost.

Fully Connected Layers

• After multiple rounds of convolutional, activation, and pooling layers, the network reaches fully connected layers.
• These layers resemble traditional neural networks, where neurons from various parts of the network are interconnected.

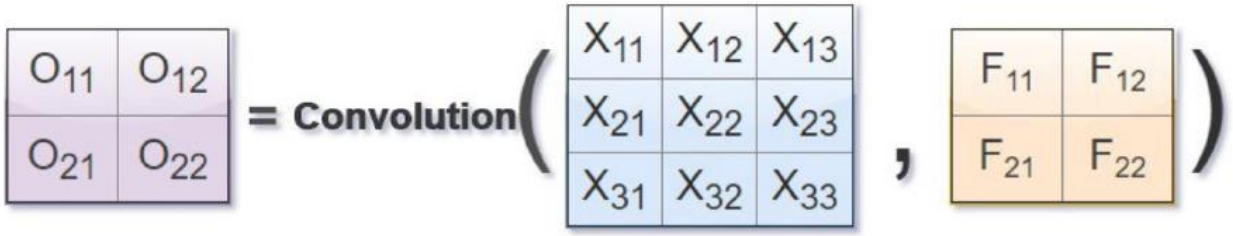


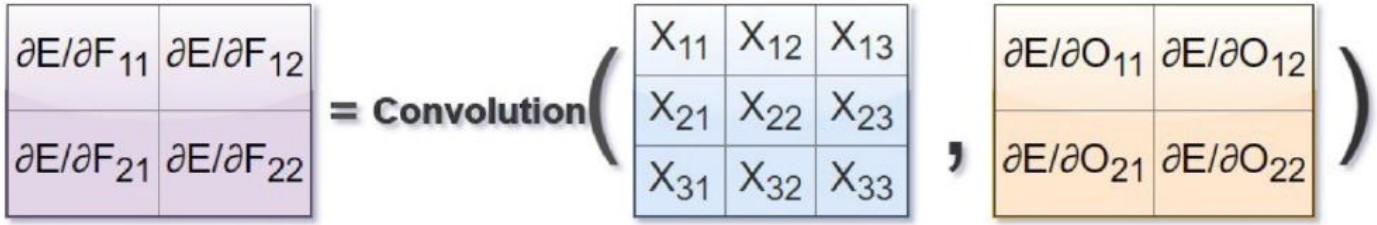
Training

Refining the Detective's Skills:

- The CNN is trained on a large dataset of labeled images.
- Each image has a corresponding label indicating the object it depicts (0,1,2..9).
- During training, the network compares its predictions with the correct labels and adjusts the weights and biases of its connections to minimize the difference.
- Imagine the detective showing his findings (extracted features) to an expert who corrects his mistakes and helps him refine his skills in identifying animals.
- Over many training iterations, the CNN progressively learns to identify even subtle features that differentiate animals. This empowers it to recognize animals in new images it hasn't encountered before.

Backpropagation of convolution





Number of learnable parameters – Input layer

- Input layer has nothing to learn
- It just provide the input image's shape.
- So no learnable parameters here. Thus number of parameters = 0.
- Parameters in general are weights that are learnt during training.
- They are weight matrices that contribute to model's predictive power, changed during back-propagation process.

Number of learnable parameters – Convolution layer

The formula for the number of parameters in a single convolutional layer is:

((kernel_width × kernel_height × input_channels) + 1) × num_filters

- **kernel_width**: The width of the convolutional filter
- **kernel_height**: The height of the convolutional filter.
- **input_channels**: The number of channels in the input data (or the number of filters from the previous layer).
- **+ 1**: Represents the bias term, which is added for each filter.
- **num_filters**: The number of filters in the current layer, which determines the number of output feature maps or channels.

Example

Consider a convolutional layer with a 3x3 kernel, 3 input channels, and 32 filters:

1. **Weights**: (3 × 3 × 3) = 27 weights per filter.
2. **Bias**: 1 bias per filter.
3. **Total per filter**: 27 + 1 = 28 parameters.
4. **Total for the layer**: 28 × 32 filters = 896 parameters.

Number of learnable parameters – Pooling layer

- This has got no learnable parameters because all it does is calculate a specific number, no backprop learning involved!
- Thus number of parameters = 0.

Number of learnable parameters – For Fully Connected (Dense) Layers

The number of parameters in a fully connected layer is calculated by:

(number of neurons in previous layer × number of neurons in current layer) + number of biases

- **number of neurons in previous layer**: The total number of flattened features from the preceding layer.
- **number of neurons in current layer (C)**: The number of units in the current fully connected layer.
- **number of biases (1*C)** : Equal to the number of neurons in the current layer.

Example

If a fully connected layer has 256 neurons from the previous layer and 4096 neurons in the current layer:

1. **Weights**: 256 × 4096 = 1,048,576.
2. **Biases**: 4096.
3. **Total for the layer**: 1,048,576 + 4096 = 1,052,672 parameters.

SL.No		Activation Shape	Activation Size
1.	Input Layer:	(32, 32, 3)	3072
2.	CONV1 (f=5, s=1)	(28, 28, 8)	6272
3.	POOL1	(14, 14, 8)	1568
4.	CONV2 (f=5, s=1)	(10, 10, 16)	1600
5.	POOL2	(5, 5, 16)	400
6.	FC3	(120, 1)	120
7.	FC4	(84, 1)	84
8.	Softmax	(10, 1)	10

1. The first **input layer** has no parameters.
2. Parameters in the second **CONV1(filter shape =5*5, stride=1) layer is: ((shape of width of filter*shape of height filter*number of filters in the previous layer+1)*number of filters) = (((5*5*3)+1)*8) = 608.**
3. The third **POOL1 layer** has no parameters.
4. Parameters in the fourth **CONV2(filter shape =5*5, stride=1) layer is: ((shape of width of filter * shape of height filter * number of filters in the previous layer+1) * number of filters) = (((5*5*8)+1)*16) = 3216.**
5. The fifth **POOL2 layer** has no parameters.

6. Parameters in the Sixth **FC3 layer is((current layer c*previous layer p)+1*c) = 120*400+1*120= 48120.**

7. Parameters in the Seventh **FC4 layer is: ((current layer c*previous layer p)+1*c) = 84*120+1* 84 = 10164.**

8. The Eighth **Softmax layer has ((current layer c*previous layer p)+1*c) parameters = 10*84+1*10 = 850.**

SL.No		Activation Shape	Activation Size	# Parameters
1.	Input Layer:	(32, 32, 3)	3072	0
2.	CONV1 (f=5, s=1)	(28, 28, 8)	6272	608
3.	POOL1	(14, 14, 8)	1568	0
4.	CONV2 (f=5, s=1)	(10, 10, 16)	1600	3216
5.	POOL2	(5, 5, 16)	400	0
6.	FC3	(120, 1)	120	48120
7.	FC4	(84, 1)	84	10164
8.	Softmax	(10, 1)	10	850

- **ImageNet Large Scale Visual Recognition Challenge** is to create model that can correctly classify an input image into 1000 separate object categories

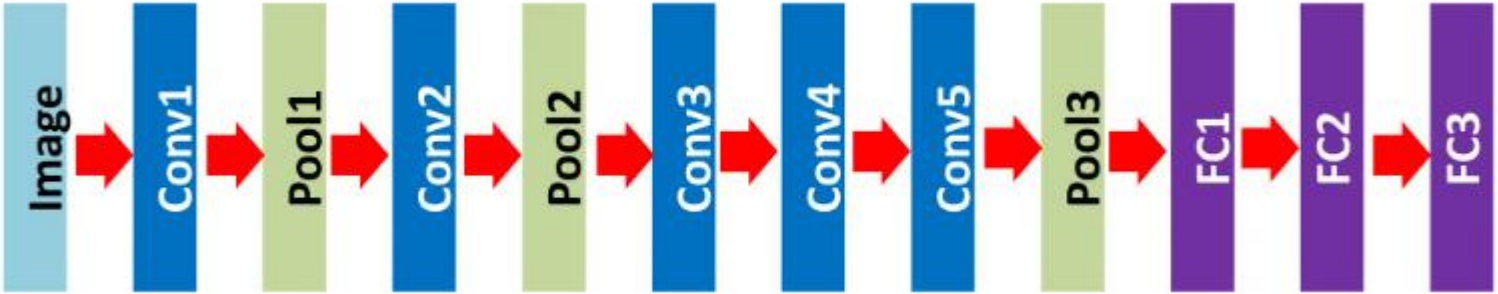
- Models are trained on 1.2 million training images with another 50000 images for validation and 1,50,000 images for testing

AlexNet

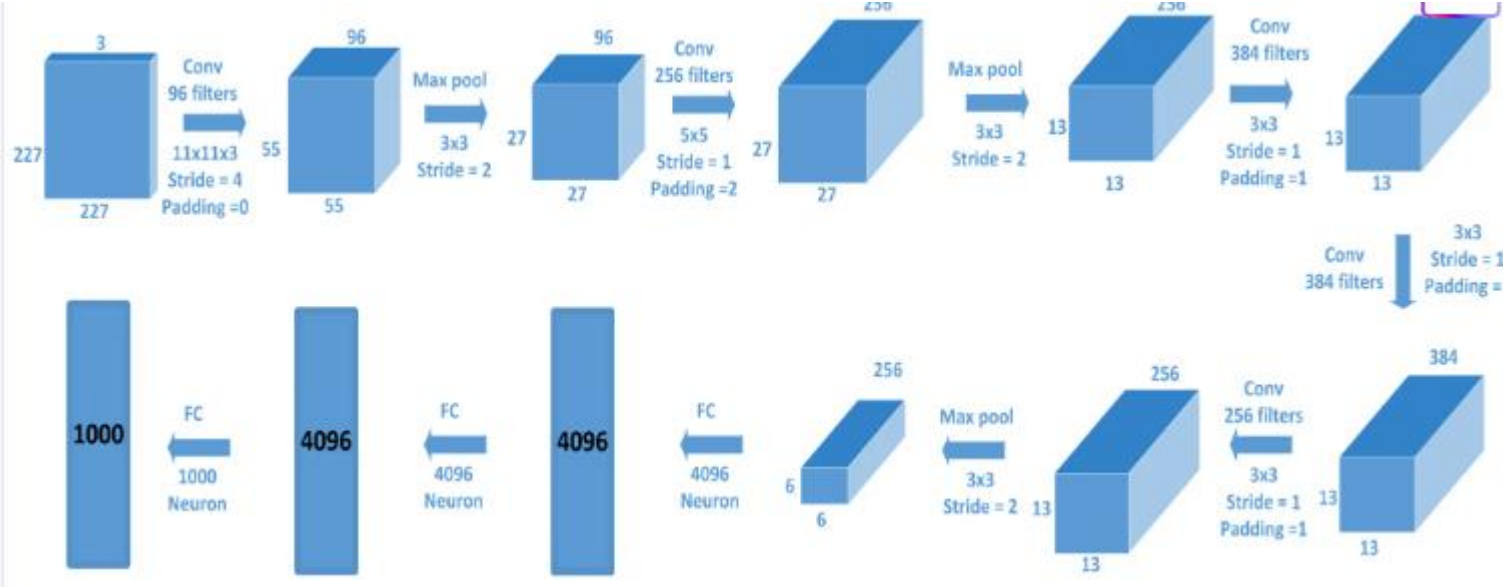
- AlexNet achieve on ILSVRC 2012 competition 15.3% top-5 error rate compared to 26.2% achieved by the second best entry
- AlexNet using Batch Stochastic Gradient Descent on training, with specific values for momentum and weight decay
- AlexNet implement dropout layers in order to combat the problem of overfitting to the training data

AlexNet

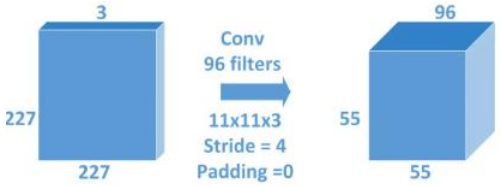
- **AlexNet** has 8 layers without count pooling layers.
- **AlexNet** use ReLU for the nonlinearity functions
- **AlexNet** trained on two GTX 580 GPUs for **five to six days**



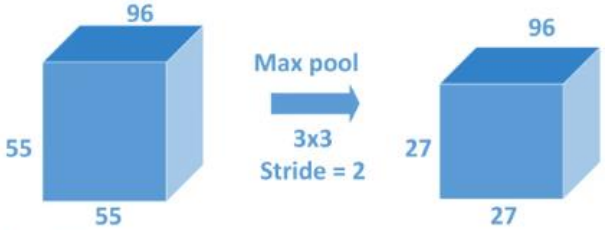
AlexNet


3
227
227
Conv
96 filters
11x11x3
Stride = 4
Padding =0
96
55
55
Max pool
3x3
Stride = 2
96
27
27
Conv
256 filters
5x5
Stride = 1
Padding =2
256
27
27
Max pool
3x3
Stride = 2
256
13
13
Conv
384 filters
3x3
Stride = 1
Padding =1
13
13
Conv
384 filters
3x3
Stride = 1
Padding =
1000
FC
1000
Neuron
4096
FC
4096
Neuron
4096
FC
4096
Neuron
256
6
6
Max pool
3x3
Stride = 2
256
13
13
Conv
256 filters
3x3
Stride = 1
Padding =1
384
13
13

AlexNet

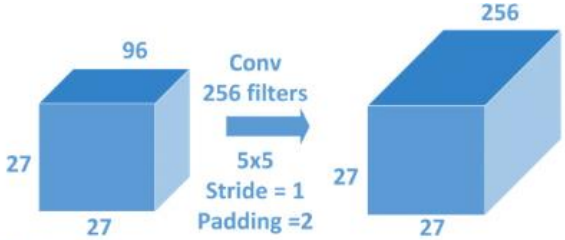


- Layer 0: 227 x 227 x 3
- Layer 1: Convolution with 96 filters, size 11×11, stride 4, padding 0
- Outcome Size= 55 x 55 x 96
- (227-11)/4 + 1 = 55 is size of outcome
- Memory: 55 x 55 x 96 x 3 (because of ReLU & LRN(Local Response Normalization))
- Weights (parameters) : 11 x 11 x 3 x 96

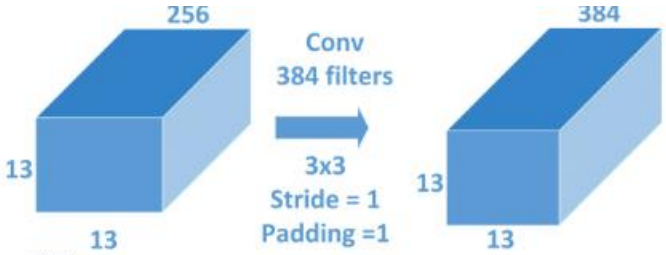


- Layer 1: 55 x 55 x 96
- Layer 2: Max-Pooling with 3×3 filter, stride 2
- Outcome Size= 27 x 27 x 96
- (55 – 3)/2 + 1 = 27 is size of outcome
- Memory: 27 x 27 x 96

AlexNet



- Layer 2: 27 x 27 x 96
- Layer 3: Convolution with 256 filters, size 5×5, stride 1, padding 2
- Outcome Size = 27 x 27 x 256
- Original size is restored because of padding
- Memory: 27 x 27 x 256 x 3 (because of ReLU and LRN)
- Weights: 5 x 5 x 96 x 256



- Layer 4: 13 x 13 x 256
- Layer 5: Convolution with 384 filters, size 3×3, stride 1, padding 1
- Outcome Size = 13 x 13 x 384
- the original size is restored because of padding (13+2 -3)/1 +1 =13
- Memory: 13 x 13 x 384 x 2 (because of ReLU)
- Weights: 3 x 3 x 256 x 384

- Layer 5: 13 x 13 x 384
- Layer 6: Convolution with 384 filters, size 3×3, stride 1, padding 1
- Outcome Size = 13 x 13 x 384
- the original size is restored because of padding
- Memory: 13 x 13 x 384 x 2 (because of ReLU)
- Weights: 3 x 3 x 384 x 384

- Layer 6: 13 x 13 x 384
- Layer 7: Convolution with 256 filters, size 3×3, stride 1,
- Outcome Size = 13 x 13 x 256
- the original size is restored because of padding
- Memory: 13 x 13 x 256 x 2 (because of ReLU)
- Weights: 3 x 3 x 384 x 256

- Layer 7: 13 x 13 x 256
- Layer 8: Max-Pooling with 3×3 filter, stride 2
- Outcome Size = 6 x 6 x 256
- (13 – 3)/2 + 1 = 6 is size of outcome
- Memory: 6 x 6 x 256

- Layer 8: 6x6x256=9216 pixels are fed to FC
- Layer 9: Fully Connected with 4096 neuron
- Memory: 4096 x 3 (because of ReLU and Dropout)
- Weights: 4096 x (6 x 6 x 256)

- Layer 9: Fully Connected with 4096 neuron
- Layer 10: Fully Connected with 4096 neuron
- Memory: 4096 x 3 (because of ReLU and Dropout)
- Weights: 4096 x 4096

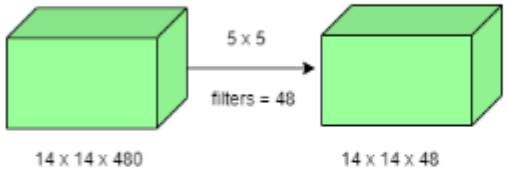
- Layer 10: Fully Connected with 4096 neuron
- Layer 11: Fully Connected with 1000 neurons
- Memory: 1000
- Weights: 4096 x 1000
- first use of ReLU
- Alexnet used Norm layers
- Alexnet heavy used data augmentation
- Alexnet use dropout 0.5
- Alexnet batch size is 128
- Alexnet used SGD Momentum 0.9
- Alexnet used learning rate 1e-2, reduced by 10

GoogleNet/ Inception V1

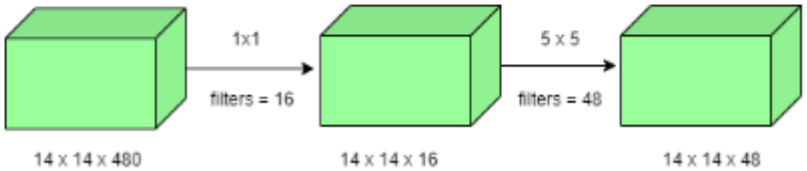
- GoogleNet is the winner of the comprtition ILSVRC 2014 with &% top-5 error rate
- GoogleNet trained on “a few high end GPUs with in a week”
- 10 times fewer parameters than AlexNet
- Much much deeper than previous CNNS
- This is made possible by the subnetwork called inception modules
- Roughly 6 million parameters

- **GoogleNet** used 9 Inception modules in the whole architecture
- This 1x1 convolutions (bottleneck convolutions) allow control/reduce the depth dimension which greatly reduces number of used parameters due to removal of redundancy correlated filters.
- **GoogleNet** has 22 Layers deep network

- **Without 1×1 Convolution:**(14×14×48)×(5×5×480)=112.9M operations

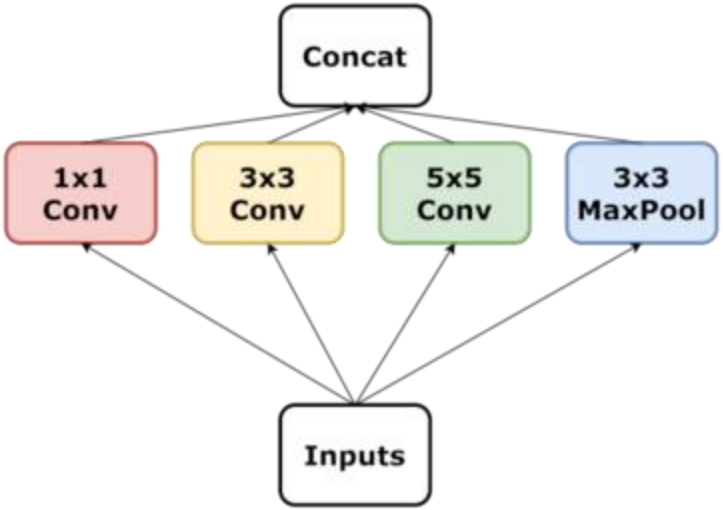


- **With 1×1 Convolution:**(14×14×16)×(1×1×480)+(14×14×48)×(5×5×16)=5.3M operations

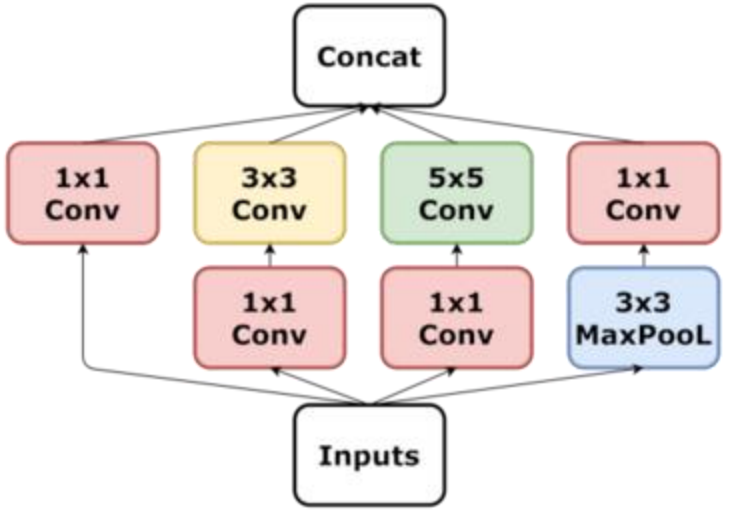


This results in a massive reduction in computation without compromising performance.

- GoogleNet use inexpensive Conv1 to compute reduction before expensive conv3 and conv5
- Conv1 follow by Relu to reduce overfitting



Inception Module



Inception Module with Dimension Reduction

Type	Size/ Stride	Output	Depth	Conv1	# Conv3	Conv3	# Conv5	Conv5	Pool	Param
Conv	7x7/2	112x112x64	**1**	-	-	-	-	-	-	2.7K
Maxpool	3x3/2	56x56x64	**0**	-	-	-	-	-	-	-
Conv	3x3/1	56x56x**192**	**2**	-	64	**192**	-	-	-	112K
Maxpool	3x3/2	28x28x192	**0**	-	-	-	-	-	-	-
Inception 3a	-	28x28x**256**	**2**	**64**	96	**128**	16	**32**	**32**	159K
Inception 3b	-	28x28x**480**	**2**	**128**	128	**192**	32	**96**	**64**	380K
Maxpool	3x3/2	14x14x480	**0**	-	-	-	-	-	-	-
Inception 4a	-	14x14x**512**	**2**	**192**	96	**208**	16	**48**	**64**	364K
Inception 4b	-	14x14x**512**	**2**	**160**	112	**224**	24	**64**	**64**	437K
Inception 4c	-	14x14x**512**	**2**	**128**	128	**256**	24	**64**	**64**	463K
Inception 4d	-	14x14x**528**	**2**	**112**	144	**288**	32	**64**	**64**	580K

Type	Size/ Stride	Output	Depth	Conv1	# Conv3	Conv3	# Conv5	Conv5	Pool	Param
Inception 4e	-	14x14x832	2	256	160	320	32	128	128	840K
Maxpool	3x3/2	7x7x832	0	-	-	-	-	-	-	-
Inception 5a	-	7x7x832	2	256	160	320	32	128	128	1072K
Inception 5b	-	7x7x1024	2	384	192	384	48	128	128	1388K
Avgpool	7x7/1	1x1x1024	0	-	-	-	-	-	-	-
Dropout .4	-	1x1x1024	0	-	-	-	-	-	-	-
Linear	-	1x1x1024	1	-	-	-	-	-	-	1000K
Softmax	-	1x1x1024	0	-	-	-	-	-	-	-

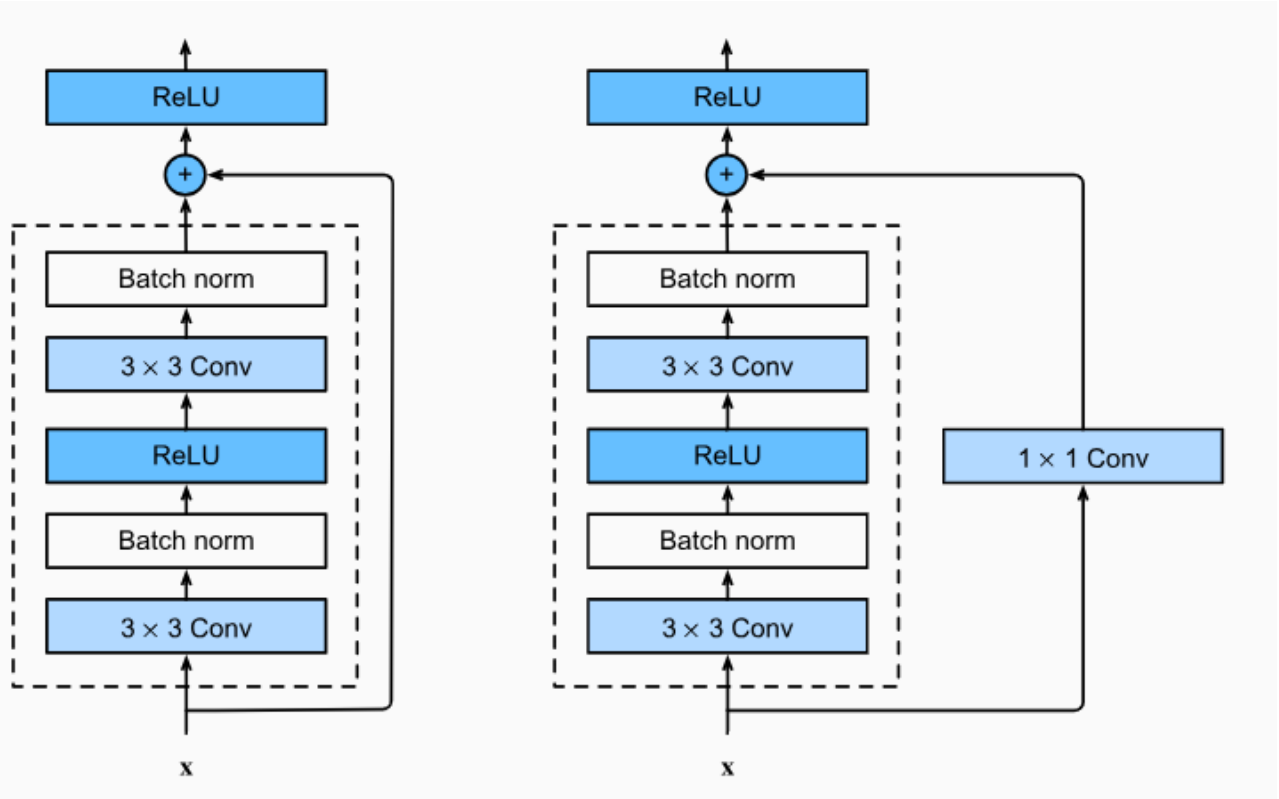
Total Layers	22

ResNet

- ResNet the winner of the competition ILSVRC 2015 with 3.6% top-5 error rate
- Proposed a residual learning approach to ease the difficulty of training deeper networks,
- Based on the design ideas of Batch Normalization, small convolutional kernels

- Residual network
- Keys:
  - No max pooling
  - No hidden fully connected layers
  - No dropout
  - Basic design (VGG style)
  - All 3x3 conv (almost)
  - Batch Normalization

ResNet



ResNet block with and without 1×1 convolution, which transforms the input into the desired shape for the addition operation.

Residual Blocks

- The residual block has two 3x3 convolutional layers with the same number of output channels.
- Each convolutional layer is followed by a batch normalization layer and a ReLU activation function.
- Then, we skip these two convolution operations and add the input directly before the final ReLU activation function.
- This kind of design requires that the output of the two convolutional layers has to be of the same shape as the input, so that they can be added together.
- If we want to change the number of channels, we need to introduce an additional 1x1 convolutional layer to transform the input into the desired shape for the addition operation.

ResNet

1. Vanishing/Exploding Gradient Problem

- As networks get very deep, gradients can vanish (become too small) or explode during backpropagation.
- Skip connections provide a **shortcut path** for the gradient to flow directly backward, reducing vanishing gradient issues and making very deep networks trainable.

2. Identity Mapping (Ease of Optimization)

- Instead of learning the full mapping H(x), ResNet reformulates it as:

  $H(x) = F(x) + x$

  where F(x) is the residual function (the part the network needs to learn).

- Learning residuals is often easier than learning the full mapping, especially when the best solution is close to the identity function.

3. Better Feature Propagation

- Information from earlier layers can be reused in deeper layers without being destroyed or transformed too much.
- This helps preserve low-level features (edges, textures) that may still be relevant for final classification.

4. Improved Generalization

- By easing optimization and stabilizing gradients, ResNets generalize better than plain deep networks.
- This is why very deep ResNets (50, 101, 152 layers) outperform plain CNNs of similar depth.

5. Mitigation of the Degradation Problem

- Without skip connections, deeper networks sometimes perform **worse** than shallower ones due to optimization difficulties.
- Skip connections prevent this degradation, ensuring deeper models don't underperform shallower versions.

ResNet

- The skip connections in ResNets allow the network to learn identity mappings making it easier for the model to learn the identity function when needed.
- Skip connections helps in optimization process which leads to faster convergence and less training time.
- The skip connections in the residual networks leads to better generalization on the unseen data as the network can skip unnnecesary or irrelevant information.

Transfer learning

- It is a machine learning technique where a model trained on one task is repurposed as the foundation for a second task.
- This approach is beneficial when the second task is related to the first or when data for the second task is limited.

Why is Transfer Learning Important?

1. **Limited Data:** Acquiring extensive labelled data is often challenging and costly. Transfer learning enables us to use pre-trained models, reducing the dependency on large datasets.
2. **Enhanced Performance:** Starting with a pre-trained model which has already learned from substantial data allows for faster and more accurate results on new tasks ideal for applications needing high accuracy and efficiency.
3. **Time and Cost Efficiency:** Transfer learning shortens training time and conserves resources by utilizing existing models hence eliminating the need for training from scratch.
4. **Adaptability:** Models trained on one task can be fine-tuned for related tasks making transfer learning versatile for various applications from image recognition to natural language processing.

How Does Transfer Learning Work?

- Transfer learning involves a structured process to use existing knowledge from a pre-trained model for new tasks:
  - **Pre-trained Model:** Start with a model already trained on a large dataset for a specific task. This pre-trained model has learned general features and patterns that are relevant across related tasks.
  - **Base Model:** This pre-trained model, known as the base model, includes layers that have processed data to learn hierarchical representations, capturing low-level to complex features.
  - **Transfer Layers:** Identify layers within the base model that hold generic information applicable to both the original and new tasks. These layers often near the top of the network capture broad, reusable features.
  - **Fine-tuning:** Fine-tune these selected layers with data from the new task. This process helps retain the pre-trained knowledge while adjusting parameters to meet the specific requirements of the new task, improving accuracy and adaptability.

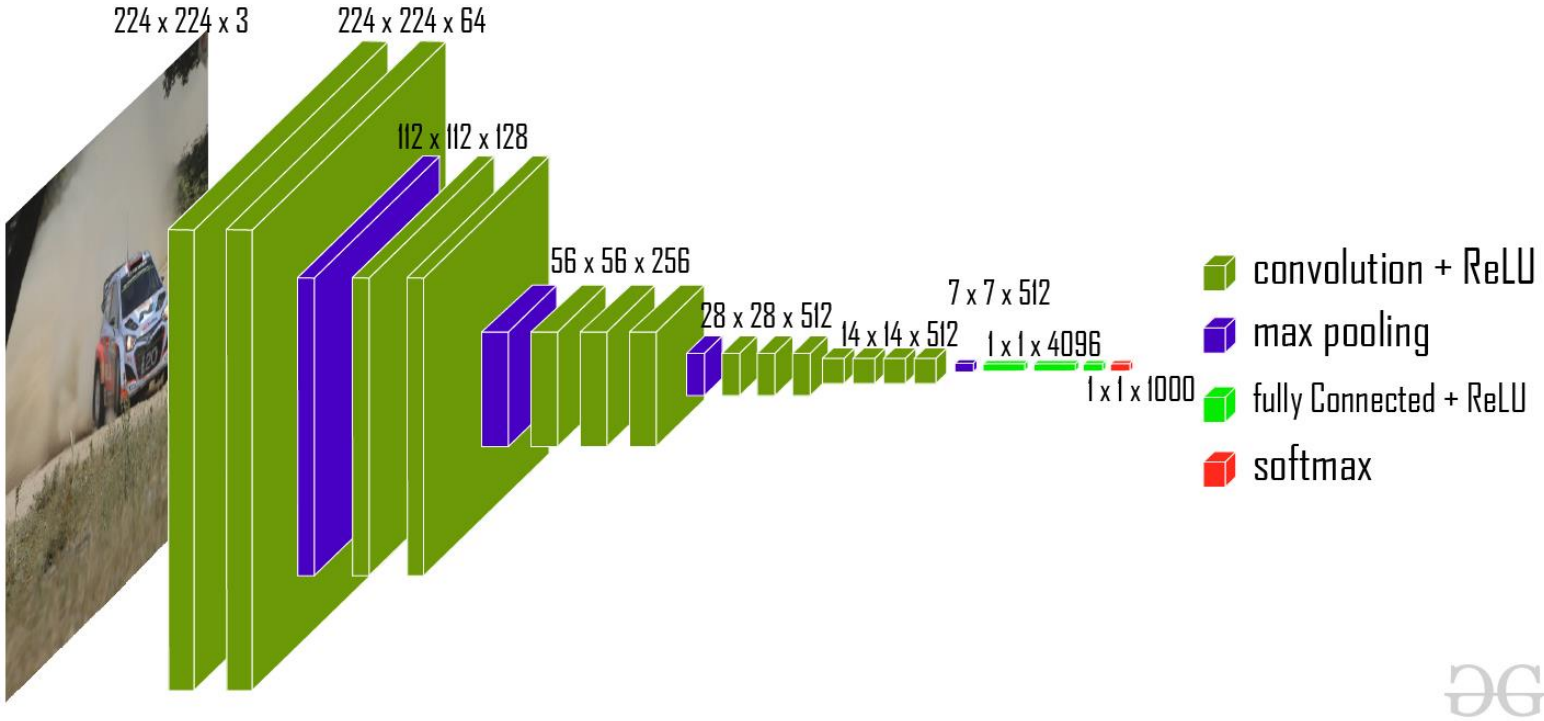
Frozen Vs Trainable Layers in Transfer Learning

- In Transfer Learning, two main components help in adapting models effectively:
  - **Frozen Layers:** These layers from a pre-trained model remain unchanged during fine-tuning. They retain general features learned from the original task, extracting universal patterns from input data.
  - **Trainable Layers:** These layers are adjusted during fine-tuning to learn task-specific features from the new dataset, allowing the model to meet the new task’s unique requirements.

How to Decide Which Layers to Freeze or Train?

- The extent to which you freeze or fine-tune layers depends on the similarity and size of your target dataset:
  - **Small, Similar Dataset:** For smaller datasets that resemble the original dataset, you freeze most layers and only fine-tune the last one or two layers to prevent overfitting.
  - **Large, Similar Dataset:** With large, similar datasets you can unfreeze more layers allowing the model to adapt while retaining learned features from the base model.
  - **Small, Different Dataset:** For smaller, dissimilar datasets, fine-tuning layers closer to the input layer helps the model learn task-specific features from scratch.
  - **Large, Different Dataset:** In this case, fine-tuning the entire model helps the model adapt to the new task while using the broad knowledge from the pre-trained model.

VGG 16


224 x 224 x 3
224 x 224 x 64
112 x 112 x 128
56 x 56 x 256
28 x 28 x 512
14 x 14 x 512
7 x 7 x 512
1 x 1 x 4096
1 x 1 x 1000
convolution + ReLU
max pooling
fully Connected + ReLU
softmax

3x3 conv, 64
3x3 conv, 64
pool/2
3x3 conv, 128
3x3 conv, 128
pool/2
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
pool/2
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool/2
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool/2
fc 4096
fc 4096
fc 4096
Size:224
Size:112
Size:56
Size:28
Size:14
Size:7
Image
Low Level Feature
Mid Level Feature
High Level Feature
Classifier

VGG 16

Layer		Feature Map	Size	Kernel Size	Stride	Activation
Input	Image	1	224 x 224 x 3	-	-	-
1	2 X Convolution	64	224 x 224 x 64	3x3	1	relu
	Max Pooling	64	112 x 112 x 64	3x3	2	relu
3	2 X Convolution	128	112 x 112 x 128	3x3	1	relu
	Max Pooling	128	56 x 56 x 128	3x3	2	relu
5	2 X Convolution	256	56 x 56 x 256	3x3	1	relu
	Max Pooling	256	28 x 28 x 256	3x3	2	relu
7	3 X Convolution	512	28 x 28 x 512	3x3	1	relu
	Max Pooling	512	14 x 14 x 512	3x3	2	relu
10	3 X Convolution	512	14 x 14 x 512	3x3	1	relu
	Max Pooling	512	7 x 7 x 512	3x3	2	relu
13	FC	-	25088	-	-	relu
14	FC	-	4096	-	-	relu
15	FC	-	4096	-	-	relu
Output	FC	-	1000	-	-	Softmax

Applications of CNN

Unveiling the Power of Convolutional Neural Networks (CNNs)

- Convolutional Neural Networks (CNNs) are a powerful type of deep learning architecture specifically designed to excel at tasks involving images and videos.
- They achieve this through a unique structure that mimics the way the human visual cortex processes information.

The Power of CNNs:

- By leveraging their unique architecture, CNNs excel at various tasks beyond image recognition. They are instrumental in:
- **Object detection:** Identifying and locating objects within images (e.g., self-driving cars detecting pedestrians).
- **Image segmentation:** Grouping pixels in an image to form meaningful segments (e.g., medical image analysis).
- **Video analysis:** Recognizing activities and objects in videos (e.g., sports analytics).

Let's delve into the world of CNNs with a detailed explanation and an image recognition example, complete with pictures!

Advantages of CNN

- **High Accuracy:** It can achieve high accuracy in various image recognition tasks.
- **Efficiency:** It is efficient, especially when implemented on GPUs.
- **Robustness:** robust to noise and variations in input data.
- **Adaptability:** It can be adapted to different tasks by modifying their architecture.

Disadvantages of CNN

- **Complexity:** It can be complex and difficult to train, especially for large datasets.
- **Resource-Intensive:** It require significant computational resources for training and deployment.
- **Data Requirements:** It need large amounts of labeled data for training.
- **Interpretability:** It can be difficult to interpret making it challenging to understand their predictions.